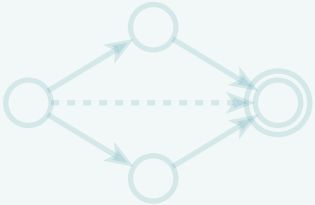
6

Лекция 16–17 марта

Недетерминированные автоматы

Константин Чухарев

Дискретная математика — Весна 2027



Недетерминизм

> *Две дороги разошлись в лесу, и я — я выбрал ту, что меньше была пройдена.*
>
> ***— Роберт Фрост***

Сколько состояний нужно?

Возьмём язык слов, у которых третий с конца символ равен 1.

Автомат должен помнить последние три символа.

Пример

Последних троек всего восемь, от 000 до 111. Тройки 100 и 000 различаются третьим с конца символом, поэтому 100 принято, а 000 нет.

Значит, эти два слова обязаны попасть в разные состояния, и так для любой пары разных троек. Состояний нужно не меньше 2^k при длине k.

Детерминизм имеет цену. Память последних k символов стоит 2^k состояний.

Недетерминизм позволяет описать тот же язык без этих состояний.

Недетерминированный конечный автомат

Вся разница с ДКА — в функции переходов. Она возвращает подмножество состояний, а слагаемое ε разрешает спонтанный шаг без чтения символа.

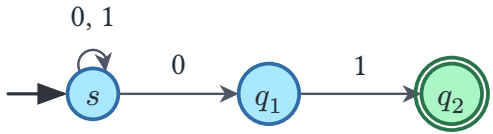
Определение 1 (НКА)

Недетерминированным конечным автоматом (НКА) называют пятёрку $M = (Q, \Sigma, \delta, q_0, F)$, где

- Q — конечное множество состояний
- Σ — конечный входной алфавит
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$ — функция переходов
- $q_0 \in Q$ — начальное состояние
- $F \subseteq Q$ — принимающие состояния

НКА принимает слово, если существует хотя бы один путь в принимающее состояние, остальные пути игнорируются.

НКА для слов, заканчивающихся на «01»



Пример

На каждом нуле автомат заподозревает начало финальной пары и проверяет догадку следующим символом. Прогон 10101 держит петлю до четвёртого символа, а последние два символа проводят путь $s \to q_1 \to q_2$ — принято.

В прогоне 10110 догадка со второго символа умирает в q_2, а последний 0 оставляет только $\{s, q_1\}$ — отвергнуто.

Прогон НКА по шагам

Переходы автомата с рисунка — $\delta(s, 0) = \{s, q_1\}$, $\delta(s, 1) = \{s\}$, $\delta(q_1, 1) = \{q_2\}$.

Прогон слова 1001 по шагам.

Прочитано	Множество состояний
ε	$\{s\}$
1	$\{s\}$
0	$\{s, q_1\}$
0	$\{s, q_1\}$
1	$\{s, q_2\}$

Финальное множество содержит $q_2 \in F$, поэтому слово принято. У слова 1000 та же траектория до последнего символа, но кончается она в $\{s, q_1\}$.

Что значит «угадывает»?

Недетерминизм часто описывают словом «угадывание».

Это слово описывает не вычисление, а краткую формулировку условия приёма.

В определении языка действует только квантор существования. Слово принято, если *существует* успешный путь.

Здесь нет ни случайности, ни параллелизма, ни алгоритма перебора. НКА — это декларативное описание языка.

Исполнительную сторону закрывают симулятор множеств и конструкция подмножеств, которые прогоняют все пути за один раз.

Сколько путей у НКА?

Каждая догадка — ветвь вычисления.

Пример (Две параллельные ветви)

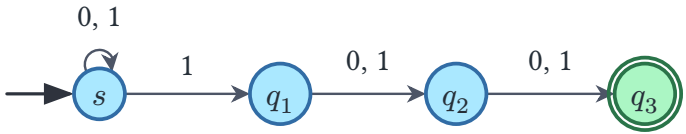
Старт s переводит нулём в два состояния сразу — в p и в q. Из p и из q каждый символ ведёт снова в оба состояния, а сами p и q — принимающие.

Слово 0^n даёт ровно 2^n путей, потому что на каждом шаге путь волен выбрать p или q. Язык при этом простой — слова, начинающиеся с нуля.

Различных множеств состояний не больше числа подмножеств Q, а путей у слова длины n бывает 2^n.

Все пути слова укладываются в одно множество, потому что для ответа важен не путь, а набор состояний, где оказались ветви.

Третий с конца символ равен 1



Пример

Догадка начинается на единице и отсчитывает три символа до конца.

Прогон 1101 даёт $\{s\} \to \{s, q_1\} \to \{s, q_1, q_2\} \to \{s, q_2, q_3\} \to \{s, q_1, q_3\}$ — принято.

Прогон 1001 даёт $\{s\} \to \{s, q_1\} \to \{s, q_2\} \to \{s, q_3\} \to \{s, q_1\}$ — отвергнуто.

НКА обходится четырьмя состояниями, а минимальному ДКА нужно $2^3 = 8$. ДКА обязан различать все хвосты длины 3.

Почему именно 8 — ответит теорема Майхилла–Нерода.

Эпсилон-переходы

ε-переход — смена состояния без чтения символа, бесплатный шаг вычисления.

Из	по a	по b	по ε
q_0	—	—	q_1
q_1	q_1	—	q_2
q_2	—	q_2	—

Пример (Разбор языка a^*b^*)

Блок a читается петлёй в q_1, блок b — петлёй в q_2, а между ними стоят ε-рёбра $q_0 \to q_1$ и $q_1 \to q_2$.

Старт — q_0, приём — q_2, и принимающий путь устроен так. ε-шаг в q_1, i петель по a, ε-шаг в q_2, j петель по b. Значит, язык равен $a^i b^j$.

Эпсилон-замыкание

ε-рёбра бесплатны, поэтому множество текущих состояний приходится замыкать.

Определение 2 (ε-замыкание)

ε-замыканием множества состояний S называют множество всех состояний, достижимых из S по цепочкам ε-переходов, включая пустую, и обозначают $\varepsilon\text{-closure}(S)$.

Пример (Замыкание обходом с очередью)

1. От q_0 ребро $q_0 \to q_1$ даёт множество $\{q_0, q_1\}$, очередь q_1.
2. От q_1 ребро $q_1 \to q_2$ даёт множество $\{q_0, q_1, q_2\}$, очередь q_2.
3. В q_2 ε-рёбер нет, очередь пуста, замыкание собрано.

Прогон слова aab проходит по множествам $\{q_0, q_1, q_2\} \to \{q_1, q_2\} \to \{q_1, q_2\} \to \{q_2\}$ — принято, конец в принимающем q_2.

Шаг чтения с эпсилон-рёбрами

Пример

Прогон слова ba по множествам даёт $\{q_0, q_1, q_2\} \to \{q_2\} \to \varnothing$.

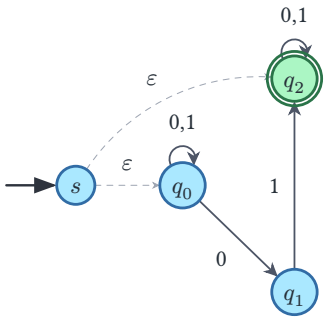
После буквы a живых путей не осталось — слово отвергнуто, дочитывать его незачем.

Шаг чтения для автомата с ε-рёбрами — всегда два действия. Сначала собрать преемников по символу из всех состояний множества, затем замкнуть результат по ε.

$$\hat{\delta}(S, a) = \varepsilon\text{-closure}\left(\bigcup_{q \in S} \delta(q, a)\right)$$

Стартовое множество тоже замыкается. Работа начинается не с $\{q_0\}$, а с ε-closure$(\{q_0\})$.

Автомат с эпсилон-ребром



Пример (Прогон)

Прогон 01 даёт путь $s \to q_0 \to q_1 \to q_2$ с шагами ε, 0, 1 — принято. Пустое слово тоже принято, ведь из s в q_2 ведёт цепочка ε-рёбер.

Неосторожный эпсилон-переход

Ребро заводили ради пустого слова, а получился приём для всего алфавита.

Пример

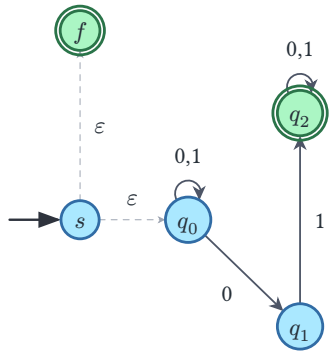
Слово 11 принимается без единой догадки. Путь $s \to q_2$ идёт по ε, а обе буквы остаются на петле в q_2.

Так же проходят 0, 0011 и любое другое слово.

Язык вырос до Σ^*

Ребро по ε из s в принимающее q_2 с петлями разрешает дочитать после него что угодно. Принимающий путь находится и для пустого слова, и для пары 01, и для любого мусора.

Пустое слово на отдельном состоянии



Пустое слово обслуживает f, куда ведёт ε-ребро из s, а исходящих рёбер у него нет. Слово 00 умирает в q_1, где ждут единицу.

Конструкция подмножеств

> *Бог не играет в кости.*
>
> ***— Альберт Эйнштейн***

Эквивалентность ДКА и НКА

Недетерминизм расширяет описание языка, но не класс распознаваемых языков.

Теорема 1 (Рабин — Скотт, 1959)

Для любого НКА существует ДКА, распознающий тот же язык.

Доказательство

Докажем построением. По НКА N соберём ДКА, состояниями которого служат подмножества Q. Начальное состояние — ε-closure$(\{q_0\})$, ведь ε-рёбра от старта бесплатны. Переход собирает всё, что может случиться за один символ.

$$\delta'(S, a) = \varepsilon\text{-closure}\left(\bigcup_{q \in S} \delta(q, a)\right).$$

Принимающие состояния — множества, пересекающие F.

Корректность конструкции подмножеств

Конструкция определена, осталось убедиться, что она принимает ровно слова НКА.

Доказательство

Докажем индукцией по длине слова. После чтения префикса w ДКА стоит в множестве всех состояний, где НКА может оказаться после w и любых ε-шагов.

База. До чтения ДКА стоит ровно в ε-closure$(\{q_0\})$, то есть во всех состояниях, настигнутых ε-рёбрами.

Шаг. Переход $\delta'(S, a)$ берёт преемников всех состояний S по a и замыкает их по ε, поэтому после wa выходит ровно множество достижимых состояний.

Слово принимается ДКА ровно тогда, когда финальное множество пересекает F, то есть когда у НКА есть успешный путь.

Тот же инвариант работает в симуляторе, который не строит ДКА заранее.

Рецепт детерминизации

Теорема обещает ДКА, рецепт собирает его руками. Множества открываются по одному, как вершины при обходе. Каждое получит строку таблицы переходов.

1. Начать с ε-замыкания начального множества $\{q_0\}$.
2. Для каждого построенного множества S и символа a собрать все переходы по a из состояний S и замкнуть результат по ε.
3. Если такое множество уже встречалось — провести дугу в него, иначе завести новое состояние.
4. Закончить, когда у всех состояний заполнены переходы.
5. Принимающими объявить множества, содержащие хотя бы одно принимающее состояние НКА.

Рецепт всегда завершается, потому что новых множеств не больше 2^n при $n = |Q|$.

Детерминизация на примере

Применим рецепт к НКА для языка «заканчивается на 01» — множества открываются по ходу обхода.

Пример (Порядок открытия)

Ведём очередь множеств и заполняем переходы, пока появляются новые.

1. Стартовое множество равно $S_0 = \{s\}$. По 0 из s расходятся два перехода, и множество $\{s, q_1\}$ оказывается новым (S_1). По 1 петля даёт $\{s\}$ — уже открыто.
2. Множество $S_1 = \{s, q_1\}$. По 0 из q_1 переходов нет, поэтому выходит снова $\{s, q_1\}$. По 1 ребро в q_2 даёт $\{s, q_2\}$ — новое (S_2).
3. Множество $S_2 = \{s, q_2\}$. По 0 выходит $\{s, q_1\}$, по 1 выходит $\{s\}$, оба множества старые. Очередь пуста, ДКА готов.

Из восьми подмножеств достижимы три, пустого среди них нет. Замыкание здесь ничего не меняет, ε-рёбер у автомата нет.

Таблица переходов ДКА

Строки идут в порядке открытия множеств.

Множество S	$\delta'(S, 0)$	$\delta'(S, 1)$
$\{s\}$	$\{s, q_1\}$	$\{s\}$
$\{s, q_1\}$	$\{s, q_1\}$	$\{s, q_2\}$
$\{s, q_2\}$	$\{s, q_1\}$	$\{s\}$

Пример

Прогон 10110 даёт $\{s\} \to \{s\} \to \{s, q_1\} \to \{s, q_2\} \to \{s\} \to \{s, q_1\}$ — отвергнуто.

Принимающее множество одно — $\{s, q_2\}$, только оно содержит q_2 из F.

Детерминизация эпсилон-автомата

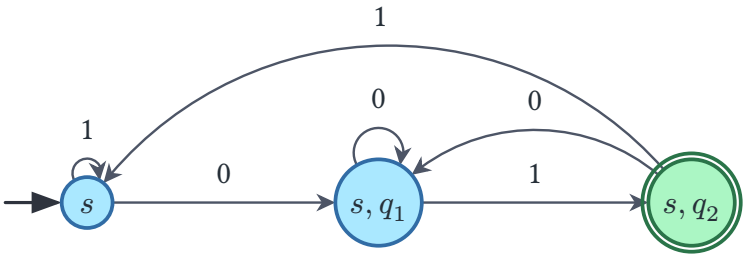
Множество S	$\delta'(S, a)$	$\delta'(S, b)$
$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_2\}$
$\{q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_2\}$
$\{q_2\}$	$\emptyset$	$\{q_2\}$
$\emptyset$	$\emptyset$	$\emptyset$

Пример (Тот же рецепт на a^*b^*)

По a из $\{q_0, q_1, q_2\}$ доступна только петля q_1, и замыкание добавляет q_2. Прогон ab даёт $\{q_0, q_1, q_2\} \to \{q_1, q_2\} \to \{q_2\}$ — принято.

Принимающие здесь — все множества, содержащие q_2. Остальные состояния не принимающие. Язык a^*b^* не замкнут относительно префикса, а $\emptyset$ — ловушка.

Множества-состояния ДКА «01»



Пример

Множества-состояния НКА, ищущего пару 01, получают роли. Множество $\{s\}$ означает ожидание, $\{s, q_1\}$ — «кончается на 0», $\{s, q_2\}$ — «кончается на 01».

Это в точности ДКА для языка «заканчивается на 01». Имена-множества играют те же роли, что q_0, q_1, q_2.

Замыкание в симуляторе НКА

Обход в глубину

Замыкание — обход в глубину по ε-рёбрам.

```rust
use std::collections::{HashMap, HashSet};

type Trans = HashMap<(&'static str, char), Vec<&'static str>>;

fn eps_closure(
    states: &HashSet<&'static str>,
    trans: &Trans,
) -> HashSet<&'static str> {
    let mut seen = states.clone();
    let mut stack: Vec<_> = states.iter().copied().collect();

    while let Some(q) = stack.pop() {
        for &r in trans.get(&(q, '\0')).into_iter().flatten() {
            if seen.insert(r) {
                stack.push(r);
            }
```

Замыкание в симуляторе НКА [2]

```
            }
        }
        seen
    }
```

Множество при замыкании только растёт, поэтому цикл завершается за линейное время.

Шаг чтения в симуляторе НКА

Шаг чтения собирает преемников по символу и сразу замыкает их.

```
fn run(
    trans: &Trans,
    start: &'static str,
    accept: &HashSet<&'static str>,
    word: &str,
) -> bool {
    let mut s = eps_closure(&HashSet::from([start]), trans);
    println!("start: {:?}", s);

    for a in word.chars() {
        let mut next = HashSet::new();
        for q in &s {
            if let Some(rs) = trans.get(&(*q, a)) {
                next.extend(rs.iter().copied());
            }
        }
        s = eps_closure(&next, trans);
```

Шаг чтения в симуляторе НКА [2]

```
            println!("{a} -> {:?}", s);
        }
        !s.is_disjoint(accept)
    }
```

Таблица ДКА здесь не строится вовсе. Множества считаются на лету, ровно для одного слова.

Прогон на «1001»

Пример

```
let ends01: Trans = HashMap::from([
    (("s", '0'), vec!["s", "q1"]),
    (("s", '1'), vec!["s"]),
    (("q1", '1'), vec!["q2"]),
]);
let accept = HashSet::from(["q2"]);
assert!(run(&ends01, "s", &accept, "1001"));
```

```
start: {"s"}
1 -> {"s"}
0 -> {"q1", "s"}
0 -> {"q1", "s"}
1 -> {"q2", "s"}
```

Прогон на «1001» [2]

Ответ `true`. Порядок элементов в `HashSet` не отсортирован, поэтому строки вывода могут идти в другом порядке. Сами множества совпадают с прогоном по таблице.

Восемь состояний из шестнадцати

Множество S	$\delta'(S,0)$	$\delta'(S,1)$
$\{s\}$	$\{s\}$	$\{s,q_1\}$
$\{s,q_1\}$	$\{s,q_2\}$	$\{s,q_1,q_2\}$
$\{s,q_2\}$	$\{s,q_3\}$	$\{s,q_1,q_3\}$
$\{s,q_1,q_2\}$	$\{s,q_2,q_3\}$	$\{s,q_1,q_2,q_3\}$
$\{s,q_3\}$	$\{s\}$	$\{s,q_1\}$
$\{s,q_1,q_3\}$	$\{s,q_2\}$	$\{s,q_1,q_2\}$
$\{s,q_2,q_3\}$	$\{s,q_3\}$	$\{s,q_1,q_3\}$
$\{s,q_1,q_2,q_3\}$	$\{s,q_2,q_3\}$	$\{s,q_1,q_2,q_3\}$

Принимающие — четыре нижние строки с q_3, и достижимы ровно эти восемь, потому что петля в s держит s в каждом.

Экспоненциальная цена детерминизма

Пример

Возьмём язык «k-й с конца символ равен 1». НКА обходится $k+1$ состояниями, а минимальный ДКА требует 2^k.

При $k = 10$ — одиннадцать состояний против 1024, при $k = 30$ — тридцать одно против $2^{30} \approx 10^9$.

Булеан конечен, поэтому детерминизация всегда заканчивается. Цену задаёт число подмножеств, а их 2^n при n состояниях.

У конечных автоматов недетерминизм устраним всегда. Для машин с неограниченной памятью тот же вопрос называется проблемой P против NP.

Сравнение ДКА и НКА

Свойство	ДКА	НКА
Переход	одно состояние	множество состояний
ε-переходы	запрещены	разрешены
Пути вычисления	ровно один	хотя бы один успешный
Дополнение языка	сменить F	сначала детерминизация
Компактность	может расти как 2^k	обычно линейная

Разница между моделями — в цене описания, а не в классе языков.

Каждый НКА превращается в ДКА, и обратное тривиально. ДКА — частный случай НКА.

Дополнение НКА

Инверсия приёма

У ДКА дополнение языка — поменять принимающие состояния местами с отвергающими.

У НКА такой приём не работает. Принятие — существование принимающего пути, а дополнение требует отсутствия *всех* принимающих путей.

Пример

Если один путь успешен, а десять нет, НКА слово принимает, а автомат с «инвертированной» F отвергнет.

Правильный путь начинается с детерминизации, и только потом меняется F. Дополнение — единственная операция над языками, которой нужен ДКА.

Недетерминизм в работе

> *Когда сомневаешься — применяй грубую силу.*
>
> ***— Кен Томпсон***

Склейки из эпсилон-переходов

Регулярное выражение собирается из автоматов-кирпичей, и все три склейки делаются ε-рёбрами.

Выражение	Склейка
$r \mid s$	новые старт и конец, ε-рёбра к обоим кирпичам и от их концов
rs	ε-ребро из конца кирпича r в старт кирпича s
r^*	ε-петля возврата в старт кирпича и ε-обход мимо цикла

Базы тоже просты. Символ a — два состояния и ребро по a, слово ε — два состояния и ε-ребро, язык $\emptyset$ — два состояния без рёбер.

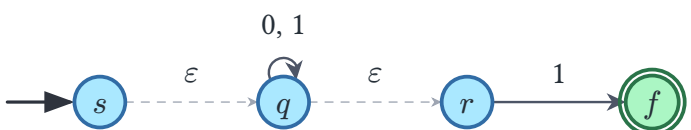
Кирпичи при склейках не перестраиваются, поэтому сборка идёт рекурсивно по дереву выражения.

Сборка автомата для $(0 \mid 1)^*1$

Пример

Соберём автомат по таблице склеек.

1. База 1 даёт пару r, f с ребром по 1.
2. Класс $0 \mid 1$ даёт q с петлями по 0 и 1.
3. Звезда добавляет старт s с ε-ребром в q и ε-выход из него.
4. Конкатенация ведёт этот выход в старт базы r, приём — f.



Звезда проверяется сразу. Пустое слово проходит по ε-рёбрам из s через q в r, но r не принимающее, поэтому для приёма нужна ещё буква 1.

Прогон автомата Томпсона

Пример

Слово 001 разбирается по автомату с рисунка.

На старте ε-closure$(\{s\}) = \{s, q, r\}$, так что цикл и его ε-выход видны ещё до первого символа.

Дальше идут два нуля подряд, и множество остаётся одним и тем же, $\{q, r\}$.

После 1 петля ведёт из q в q, ребро ведёт из r в f, а замыкание возвращает r по ε-ребру из q. Текущее множество $\{q, f, r\}$, и слово кончилось в принимающем состоянии.

Состояние r живёт во множестве с самого старта, и автомат параллельно ждёт цикл и последний символ 1.

Слово 010 отвергается. Последний 0 стирает приёмник f из текущего множества, и остаются только $\{q, r\}$, то есть ни одного принимающего.

Замкнутость через три склейки

Склейки годятся не только для сборки примера, но и для доказательства замкнутости.

Теорема 2 (Замкнутость класса НКА)

Если L_1 и L_2 распознаются НКА, то НКА распознают и $L_1 \cup L_2$, и $L_1 L_2$, и L_1^*.

Доказательство

Докажем построением на копиях автоматов с непересекающимися состояниями.

Объединение. Новый старт s ведёт ε-рёбрами в оба старта копий, а их концы — в новый принимающий, поэтому путь идёт через одну копию.

Конкатенация. Конец первой копии соединён со стартом второй, приём — конец второй, поэтому путь читает uv с $u \in L_1$, $v \in L_2$.

Звезда. Новые s, f получают ε-рёбра $s \to s_1$ и $f_1 \to f$, повтор $f_1 \to s_1$ и обход $s \to f$.

Выражение в конвейере лексера

Пример (Конвейер лексера)

Каждый токен компилятора — регулярное выражение, будь то идентификатор, число или строковый литерал.

Конструкция Томпсона собирает выражения в НКА, конструкция подмножеств детерминизирует их в ДКА, и дальше каждый символ — один табличный переход за $O(1)$.

Недетерминизм укорачивает описание, детерминизм даёт исполнение без возвратов.

Движки с возвратами (backtracking) на злых выражениях взрываются по времени, поэтому линейные движки вроде `regex` в Rust строят автомат, а не перебирают пути.

Поиск подстроки как угадывание

Пример (НКА для языка «содержит 101»)

Петли в s откладывают догадку, цепочка $s \to q_1 \to q_2 \to q_3$ проверяет шаблон, петли в q_3 дочитывают хвост.

Ключевой переход $\delta(s, 1) = \{s, q_1\}$ позволяет продолжить поиск или начать проверку.

Прогон слова 11011.

$$\{s\} \to \{s, q_1\} \to \{s, q_1\} \to \{s, q_2\}$$
$$\to \{s, q_1, q_3\} \to \{s, q_1, q_3\}$$

В финальном множестве есть q_3 — принято.

Симуляция множеств читает текст одним проходом без возвратов — недетерминированный поиск, чья битовая упаковка известна как алгоритм Shift-And.

Когда счётчик лучше угадывания?

Пример (Двоичная запись делится на 3)

О префиксе достаточно помнить остаток по модулю 3. Дописывание бита b превращает остаток r в $(2r + b) \bmod 3$.

Состояния — остатки r_0, r_1, r_2, а старт и принимающее совпадают с r_0. Переход по биту b из r_i ведёт в $r_{(2i+b) \bmod 3}$.

Прогон 110 даёт $r_0 \to r_1 \to r_0 \to r_0$ — принято, и это верно, ведь $6 = 2 \cdot 3$.

Прогон 1011 кончается в r_2, и одиннадцать на три не делится.

Угадывать здесь нечего, потому что свойство всего слова — один арифметический инвариант.

Недетерминизм окупается там, где свойство звучит как «где-то», «хотя бы одна», «одна из альтернатив».

Недетерминизм и языки

НКА распознают в точности тот же класс языков, что и ДКА.

- У НКА переход возвращает множество состояний, а слово принимается, если существует успешный путь.
- ε-переходы бесплатны, и ε-замыкание нужно учитывать на каждом шаге чтения.
- Конструкция подмножеств симулирует все пути сразу, а состояниями ДКА служат множества состояний НКА.
- Экспонента 2^k бывает настоящей, и язык «третий с конца» даёт восемь достижимых множеств из шестнадцати.
- Томпсон собирает автомат для регулярного выражения ε-склейками, и те же склейки доказывают замкнутость класса.
- Дополнение — единственная операция, требующая детерминизации. Недетерминизм устраним всегда, но ценой может быть экспонента.

Вопросы по недетерминизму

недетерминизм и детерминизация

1. Почему «угадывает» — плохое описание НКА? Дайте формулировку через квантор.
2. Сколько состояний у НКА и у минимального ДКА для языка «третий с конца равен 1»?
3. Какие два действия составляют шаг чтения с ε-переходами, и что сломается, если в конструкции подмножеств забыть замыкание?
4. Почему в детерминизации автомата «третий с конца» достижимы ровно восемь множеств из шестнадцати?
5. Перечислите склейки конструкции Томпсона. Где в каждой работает ε-ребро?
6. Почему для дополнения языка НКА мало поменять принимающие состояния?

Что почитать?

Задача	Где читать
Понять НКА	m23, раздел «Недетерминированные конечные автоматы»
Детерминизировать	m23, раздел «Конструкция подмножеств»
Строить из regex	m23, раздел «Регулярные выражения»
Разобрать лексер	m23, раздел «Лексер: регулярные выражения в компиляторе»